

21.Aim

To write and execute a **Prolog program to implement the Tower of Hanoi problem** using recursion.

Algorithm

1. Start.
2. Read the number of disks N.
3. If $N = 1$, move the disk directly from the **Source** rod to the **Destination** rod.
4. Otherwise:
 - Move N-1 disks from **Source** to **Auxiliary**.
 - Move the Nth disk from **Source** to **Destination**.
 - Move N-1 disks from **Auxiliary** to **Destination**.
5. Stop.

Program

% Tower of Hanoi

% Base case

hanoi(1, Source, Destination, _) :-

write('Move disk 1 from '),

write(Source),

write(' to '),

write(Destination), nl.

% Recursive case

hanoi(N, Source, Destination, Auxiliary) :-

N > 1,

M is N - 1,

hanoi(M, Source, Auxiliary, Destination),

write('Move disk '),

```
write(N),  
write(' from '),  
write(Source),  
write(' to '),  
write(Destination), nl,
```

```
hanoi(M, Auxiliary, Destination, Source).
```

Sample Input

?- hanoi(3, a, c, b).

Where:

- 3 = Number of disks
- a = Source rod
- c = Destination rod
- b = Auxiliary rod

Sample Output

```
Move disk 1 from a to c  
Move disk 2 from a to b  
Move disk 1 from c to b  
Move disk 3 from a to c  
Move disk 1 from b to a  
Move disk 2 from b to c  
Move disk 1 from a to c
```

true.

Result

Thus, the **Tower of Hanoi problem** was successfully implemented using Prolog recursion, and the required sequence of disk movements was obtained.

22. Aim

To write and execute a Prolog program to determine whether a particular **bird can fly or cannot fly** using facts and rules.

Algorithm

1. Start.
2. Define the names of different birds as facts.
3. Define birds that cannot fly.
4. Create a rule to identify birds that can fly.
5. Create a rule to identify birds that cannot fly.
6. Execute queries to check a particular bird.
7. Display the result.
8. Stop.

Program

% Facts

bird(parrot).

bird(eagle).

bird(pigeon).

bird(ostrich).

bird(penguin).

% Birds that cannot fly

cannot_fly(ostrich).

cannot_fly(penguin).

% Rule for birds that can fly

can_fly(X) :-

bird(X),

\+ cannot_fly(X).

% Rule for birds that cannot fly

can_not_fly(X) :-

bird(X),

cannot_fly(X).

Sample Input / Required Queries

Query 1: Check whether a parrot can fly

?- can_fly(parrot).

Sample Output

true.

Query 2: Check whether a penguin can fly

?- can_fly(penguin).

Sample Output

false.

Query 3: Check whether a penguin cannot fly

?- can_not_fly(penguin).

Sample Output

true.

Query 4: Display all birds that can fly

?- can_fly(X).

Sample Output

X = parrot ;

X = eagle ;

X = pigeon.

Query 5: Display all birds that cannot fly

?- can_not_fly(X).

Sample Output

X = ostrich ;

X = penguin.

Result

Thus, the **Prolog** program to determine whether a particular bird can fly or cannot fly was successfully implemented and executed using facts, rules, and queries.

23. Family Tree in Prolog

Aim

To implement a **family tree** using **Prolog** facts and rules.

Algorithm

1. Define family members and their gender.
2. Define parent relationships.
3. Create rules for father, mother, sibling, grandfather, etc.
4. Execute queries to find relationships.

Program

male(ram).

male(suresh).

male(raju).

female(sita).

female(lakshmi).

female(radha).

female(priya).

parent(ram, suresh).

parent(sita, suresh).

parent(ram, lakshmi).

parent(sita, lakshmi).

parent(suresh, raju).

parent(radha, raju).

parent(suresh, priya).

parent(radha, priya).

father(X,Y) :- parent(X,Y), male(X).

mother(X,Y) :- parent(X,Y), female(X).

sibling(X,Y) :-

parent(Z,X),

parent(Z,Y),

$X \neq Y$.

grandfather(X,Y) :-

parent(X,Z),

parent(Z,Y),

male(X).

Sample Queries

?- father(X, raju).

X = suresh.

?- mother(X, priya).

X = radha.

?- sibling(raju, X).

X = priya.

?- grandfather(X, raju).

X = ram.

Result

Thus, the **family tree was successfully implemented using Prolog facts, rules, and queries.**

24. Diet Suggestion System in Prolog

Aim

To write a Prolog program to **suggest a diet based on a disease.**

Algorithm

1. Define diseases as facts.
2. Define suitable diet suggestions for each disease.
3. Create a rule to find the diet based on the disease.
4. Execute the query and display the suggested diet.

Program

% Disease and diet facts

diet(diabetes, low_sugar_diet).

diet(hypertension, low_salt_diet).

diet(obesity, low_calorie_diet).

diet(anemia, iron_rich_diet).

diet(heart_disease, low_fat_diet).

% Rule to suggest diet

suggest_diet(Disease, Diet) :-

diet(Disease, Diet).

Sample Queries

?- suggest_diet(diabetes, X).

X = low_sugar_diet.

?- suggest_diet(anemia, X).

X = iron_rich_diet.

?- suggest_diet(obesity, X).

X = low_calorie_diet.

Result

Thus, the Prolog program to **suggest a suitable diet based on a disease** was successfully implemented using facts, rules, and queries.

25. Monkey Banana Problem in Prolog

Aim

To write a Prolog program to implement the **Monkey and Banana problem**, where a monkey moves, climbs a box, and gets the banana.

Algorithm

1. Place the monkey, box, and banana at different positions.
2. Move the monkey towards the box.
3. Push the box under the banana.
4. Climb onto the box.
5. Reach and get the banana.

Program

% Monkey Banana Problem

% monkey(Position, Status, HasBanana)

% Initial state

initial_state(state(door, floor, no, window)).

% Goal state

goal_state(state(_, _, yes, _)).

% Move monkey from door to box

move(state(door, floor, no, Box),

 walk_to_box,

 state(Box, floor, no, Box)).

% Push box under banana

move(state(Box, floor, no, Box),

 push_box,

 state(middle, floor, no, middle)).

% Climb onto the box

move(state(middle, floor, no, middle),

 climb_box,

 state(middle, on_box, no, middle)).

% Get the banana

move(state(middle, on_box, no, middle),

 grab_banana,

 state(middle, on_box, yes, middle)).

% Solve the problem

solve(State, []) :-

 goal_state(State).

solve(State, [Action | Rest]) :-

```
move(State, Action, NewState),  
solve(NewState, Rest).
```

Sample Query

```
?- initial_state(S), solve(S, Actions).
```

Sample Output

```
S = state(door, floor, no, window),
```

```
Actions = [walk_to_box, push_box, climb_box, grab_banana].
```

Result

Thus, the **Monkey Banana problem was successfully implemented in Prolog**, and the monkey finds the sequence of actions required to obtain the banana.